Exam Seat No. | |

Walwa Taluka Education Society's

YASHWANTRAO CHAVAN MAHAVIDYALAYA, ISLAMPUR

Tal: Walwa, Dist: Sangli.

Department Of Computer Science

LABORATORY CERTIFICATE

This is to certify that, Mr./Miss. _____

Roll No. _____ has satisfactorily completed the course of Experiment in **VSC II (Major Specific) Node.js** as per syllabus laid down by Shivaji University for **B.Sc. Part – III Computer Science Optional Sem V** during the academic year 2026-2027.

Date: / / 2026

Teacher in charge **Examiner** **Head**

Department of Computer Scienc

INDEX

VSC II (Major Specific) Node.js

Sr. No.	Title	Page No.	Date	Remark
1	Write a Node.js script to display the current Node.js version and operating system platform using the process object.		08-07-2026	
2	Create a Buffer from the string ”Node.js” and convert it back to string after displaying the raw buffer content.		08-07-2026	
3	Write a Node.js program that prints “Hello” immediately and “World” after a 3-second delay using set Timeout().		08-07-2026	
4	Create an HTTP server that listens on part 3000 and display ” Server Running” in the browser		15-07-2026	
5	Write an HTTP server that displays “Home Page ”for/home route and “About Page” for/about route		15-07-2026	
6	create a simple API endpoint /user that returns a JSON object with name,age and city fields.		22-07-2026	
7	Extract the name and age parameter from the query string “name=John&age =25”using the querystring module		22-07-2026	
8	Extract the name and age parameter from the query string “name=John & age =25”using the query string module.		22-07-2026	
9	Create a custom module math.js that exports an add(a, b)function and use it in another file app.js.		05-08-2026	
10	Write a Node.js script to display the current working directory and memory usage using global process object		05-08-2026	
11	Create an HTTP server that reads the name parameter from URl query and display “Hello [name]”in the browser.		12-08-2026	
12	Install and use the chalk module to print “Success” in green color and “Error” in red color.		12-08-2026	
13	Create an HTTP server that handles both GET and POST Requests and displays appropriate messages for each method		19-08-2026	
14	Write an Eventmitter program where a listener receives arguments(name and age) and displays them when an ”info” event is emitted.		02-09-2026	
15	Create an event listener that runs only once using the once()method and verify that it doesn’t execute on second emit		09-09-2026	
16	Write a Node.js program to connect to MongoDB and insert a single document into a “students”collection		23-09-2026	

Practical No: 1

Q1) Write a Node.js script to display the current Node.js version and operating system platform using the process object.

Program Analysis:

To write a Node.js program to display the current Node.js version and operating system platform using the process object.

Program:

```
console.log ("Node.js Version:", process. version);
console.log ("Operating System Platform:", process. platform);
```

Output:

```
Node.js Version: v22.14.0
Operating System Platform: win32
```

Practical No: 2

Q2) Create a Buffer from the string ”Node.js” and convert it back to string after displaying the raw buffer content.

Program Analysis:

To create a Buffer from the string "Node.js", display the raw buffer content, and convert the Buffer back into a string.

Program:

```
const buffer = Buffer. from ("Node.js");
console. log("Raw Buffer:", buffer);
const str = buffer. toString();
console.log("Converted String:", str);
```

Output:

```
Raw Buffer: <Buffer 4e 6f 64 65 2e 6a 73>
Converted String: Node.js
```

Practical No: 3

Q3) Write a Node.js program that prints “Hello” immediately and “World” after a 3-second delay using set Timeout().

Program Analysis:

To write a Node.js program that prints “Hello” immediately and “World” after a 3-second delay using set Timeout().

Program:

```
console.log("Hello");
setTimeout(() => {console.log("World"); }, 3000);
```

Output:

```
Hello
World
```

Practical No: 4

Q4) Create an HTTP server that listens on part 3000 and display ” Server Running” in the browser.

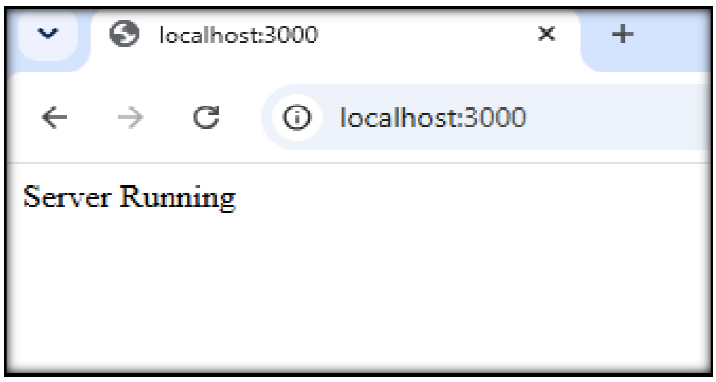
Program Analysis:

To create a Node.js HTTP server using the built-in http module.
To make the server listen on port 3000.
To display “Server Running” in the browser.
To understand the basic working of an HTTP server.

Program:

```
const http = require("http");
const server = http.createServer((req, res) => { res.writeHead(200, {"Content-Type": "text/html" }); res.end("Server Running");});
server.listen(3000, () => {
  console.log("Server is running on port 3000");
});
```

Output:



Practical No: 5

Q5) Write an HTTP server that displays “Home Page ”for/home route and “About Page” for/about route.

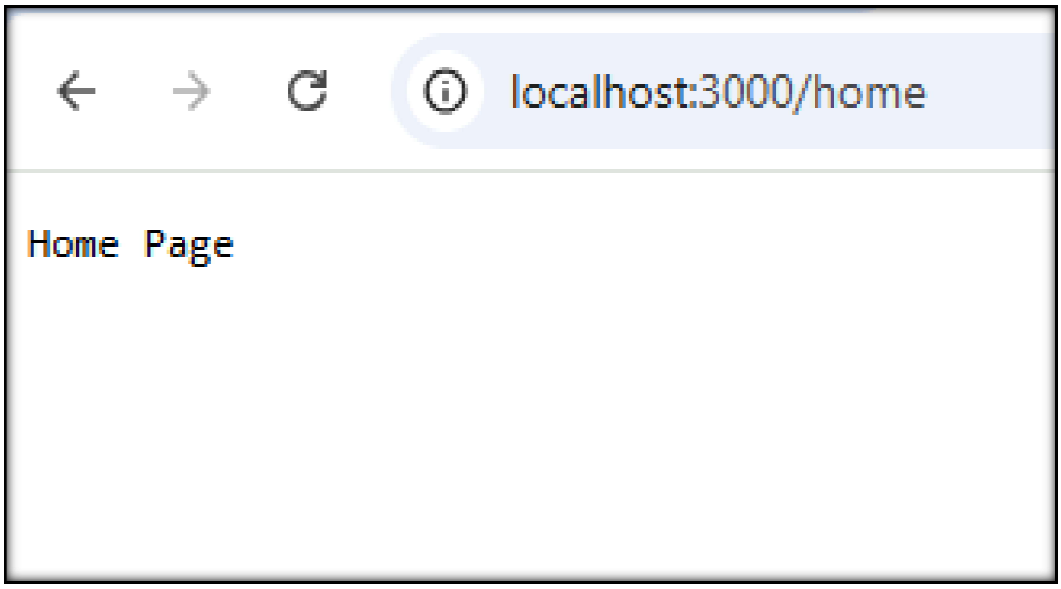
Program Analysis:

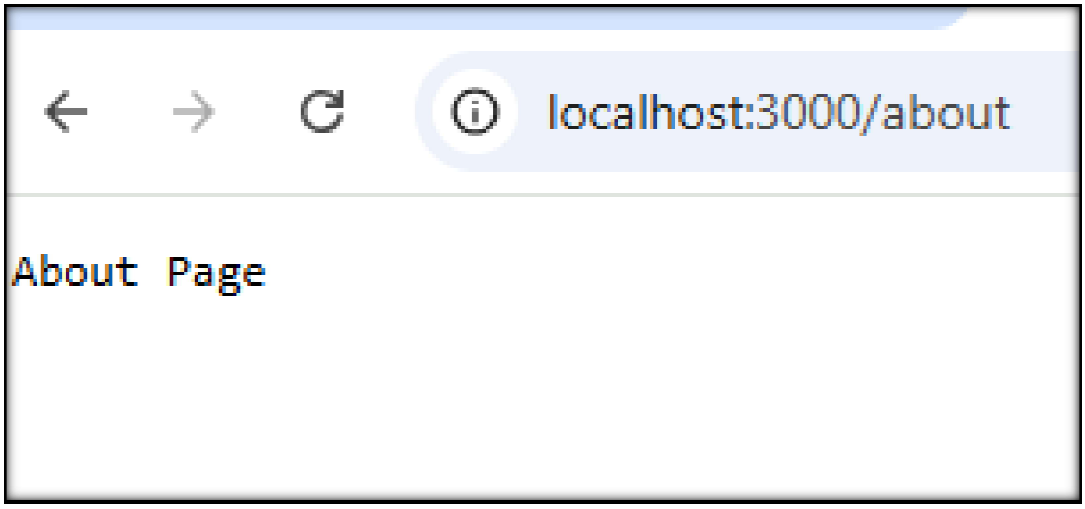
This Node.js program creates a simple HTTP server using the built-in http module. The http. create Server() method is used to create the server, and the req.url property is used to check the requested URL. When the user visits the / route, the server displays "Home Page", and when the user visits the /about route, it displays "About Page".

Program :

```
const http = require("http");
const server = http.createServer((req, res) => {
res.writeHead(200, {"Content-Type": "text/html"});
if (req.url == "/home") {
res.end("<h1>Home Page</h1>");
 } else if (req.url == "/about") {
 res.end("<h1>About Page</h1>");
  }
else
   {
res.end("<h1>Page Not Found</h1>");
      }
    });
   server.listen(3000, () => {
 console.log("Server running...");
});
```

Output:


localhost:3000/home
Home Page


localhost:3000/about
About Page

Practical No: 6

Q6) create a simple API endpoint /user that returns a JSON object with name,age and city fields.

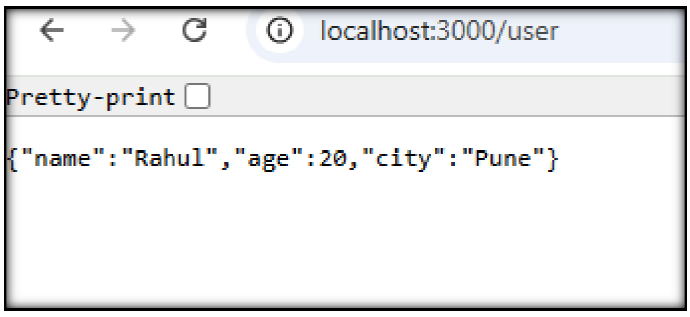
Program Analysis:

The http module is used to create an HTTP server. The req.url == "/user" condition is used to check the /user endpoint. A JSON object named user is created with three fields: name, age, and city. The JSON.stringify(user) method converts the object into a JSON string, and res.end() sends the JSON response to the client. If a URL other than /user is requested, the server displays Page Not Found. The server runs on port 3000**.**

Program:

```
 const http = require("http");
 const server = http.createServer((req, res) => {
 if (req.url == "/user") {
res.writeHead(200, { "Content-Type": "application/json" });
 const user = {
name: "Rahul",
age: 20,
city: "Pune"
  };
 res.end(JSON.stringify(user));
 }
 else {
  res.writeHead(404, { "Content-Type": "text/plain" });
  res.end("Page Not Found");
 }
 });
 server.listen(3000, () => {
console.log("Server running at http://localhost:3000");
 });
```

Output:



Practical No: 7

Q7) Write a program using the url module to parse URl http://localhost/profile?id=101 and display the pathname and query string.

Program Analysis:

The url module is used to parse the given URL. The url.parse() method extracts different parts of the URL. The pathname property displays /profile, while the query property displays id=101. Thus, the program displays the pathname and query string of the given URL.

Program:

```
const url = require("url");
const myUrl = "http://localhost/profile?id=101";
const parsedUrl = url.parse(myUrl);
console.log("Pathname:", parsedUrl.pathname);
console.log("Query String:", parsedUrl.query);
```

Output:

```
Pathname: /profile
Query String: id=101
```

Practical No: 8

Q8) Extract the name and age parameter from the query string “name=John & age =25”using the query string module.

Program Analysis:

The query string module is used to parse the query string. The query string. parse() method converts name=John & age=25 into an object. The result.name gives the value John**,** and result. age gives the value 25**.** Thus, the program extracts the name and age parameters from the query string.

Program:

```
const querystring = require("querystring");
const data = "name=John&age=25";
const result = querystring.parse(data);
console.log("Name:", result.name);
console.log("Age:", result.age);
```

Output:

```
Name: John
Age: 25
```

Practical No: 9

Q9) Create a custom module math.js that exports an add(a, b)function and use it in another file app.js.

Program Analysis:

The math.js file is a custom module that contains an add() function. The module. exports statement exports this function so it can be used in another file. In app.js, the require("./math") statement imports the add() function from math.js. The function is called with 10 and 20, and the result is displayed using console.log().

Program:

1) **Math.js**

```
function add(a, b) {
  return a + b;
  }
module. exports = add;
```

2) **app.js**

```
const add = require("./math");
const result = add(10, 20);
 console.log("Addition:", result);
```

Output:

```
Addition:30
```

Practical No:10

Q10)Write a Node.js script to display the current working directory and memory usage using global process object.

Program Analysis:

The global process object is used to get information about the Node.js program. The process.cwd() method displays the current working directory. The process.memory Usage() method displays the memory used by the Node.js process, such as RSS, heap total, heap used, and external memory.

Program:

```
console.log("Current Working Directory:", process.cwd());
console.log("Memory Usage:");
 console.log(process.memoryUsage());
```

Output:

```
Current Working Directory: C:\Users\Admin\Desktop\project node
Memory Usage:
{
rss: 35274752,
heapTotal: 7118848,
heapUsed: 4929416,
external: 1998487,
arrayBuffers: 137503
}
```

Practical No: 11

Q11)Create an HTTP server that reads the name parameter from URl query and display “Hello [name]”in the browser.

Program Analysis:

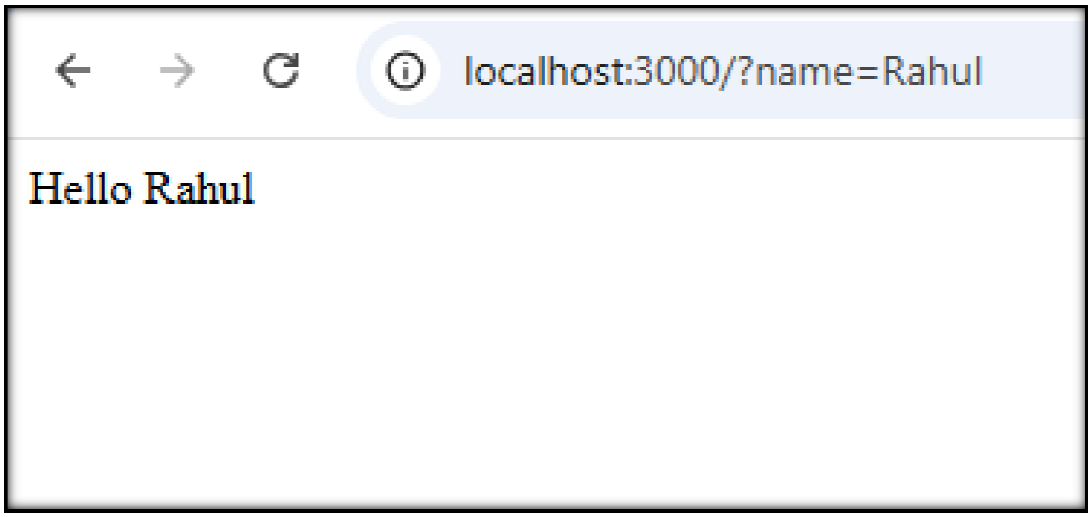
The http module is used to create an HTTP server, while the url module is used to read the query parameter from the URL. The url.parse() method separates the URL into different parts, and data.query.name gets the value of the name parameter. If the name is present, the server displays Hello [name] in the browser. Otherwise, it displays Please enter name. The server runs on port 3000

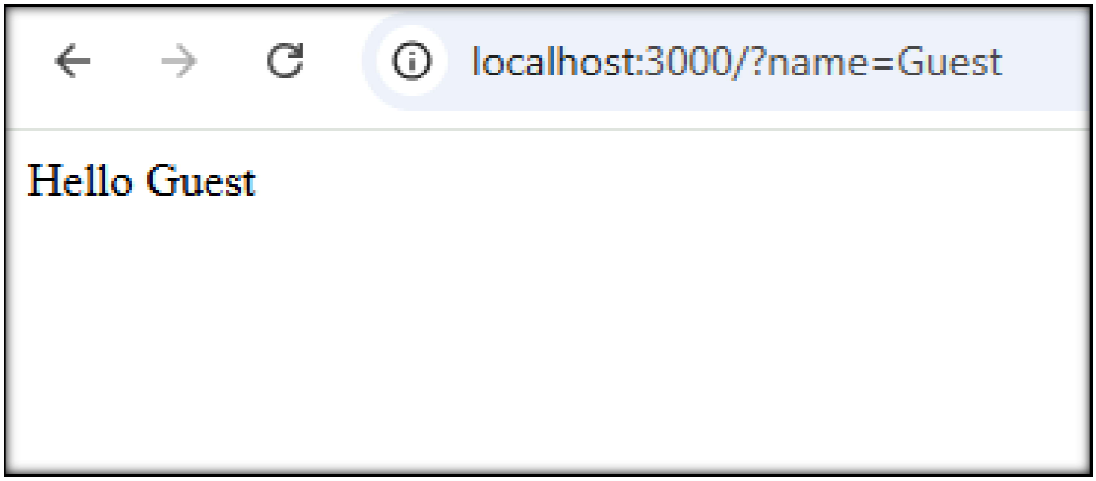
Program:

```
const http = require('http');
 const url = require('url');
 const server = http.createServer((req, res) => {
const query = url.parse(req.url, true).query;
const name = query.name || 'Guest';
res.writeHead(200, { 'Content-Type': 'text/html' });
res.end(`Hello ${name}`);
});

 server.listen(3000, () => {
console.log('Server running at http://localhost:3000');
});
```

Output:


localhost:3000/?name=Rahul
Hello Rahul


localhost:3000/?name=Guest
Hello Guest

Practical No: 12

Q12) Install and use the chalk module to print “Success”in green color and “Error” in red color

Program Analysis:

This program used to chalk modules to display messages in different colors.

Program:

```
const chalk = require('chalk');
 console.log(chalk.green('Success'));
console.log(chalk.red('Error'));
```

Output:

```
C:\Users\Admin\Desktop\project node>node -v
v24.21.0
 C:\Users\Admin\Desktop\project node>npm -v
11.19.0
C:\Users\Admin\Desktop\project node>npm install chalk

up to date, audited 19 packages in 1s

2 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\Admin\Desktop\project node>node app.js
Success
Error
```

Practical No: 13

Q13) Create an HTTP server that handles both GET and POST Requests and displays appropriate messages for each method.

Program Analysis:

This program creates an HTTP server that checks the request method. If the method is GET, it displays a GET message. If the method is POST, it displays a POST message. For any other method, it displays Method not supported.

Program:

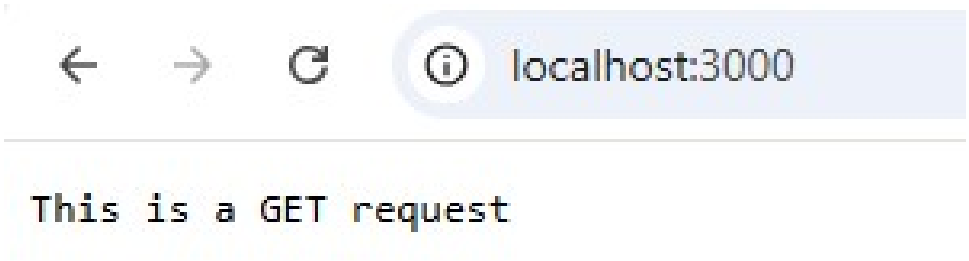
1) **GET Request:**

```
const http = require('http');
http.createServer((req, res) => {
if (req.method === 'POST') {
res.end('POST Request');
} else {
res.end('GET Request');
}
}).listen(3000);
console.log('Server started');
```

2) **POST Request:**

```
const http = require('http');
const req = http.request({
hostname: 'localhost',
port: 3000,
method: 'POST'
}, (res) => {
res.on('data', (data) => {
console.log(data.toString());
});
});
req.end();
```

Output:



PS C:\Users\Admin\Desktop\project node> node post.js

POST Request

Practical No: 14

Q14)Write an Eventmitter program where a listener receives arguments(name and age)and displays them when an”info”event is emitted.

Program Analysis:

The program uses Node.js EventEmitter to create an "info" event. A listener receives the name and age arguments and displays them. The emit() method triggers the event and passes the values to the listener.

Program:

```
const EventEmitter = require('events');
const emitter = new EventEmitter();
// Listener function receives name and age
emitter.on('info', (name, age) => {
    console.log("Name:", name);
    console.log("Age:", age);
});
// Emit the "info" event with arguments
emitter.emit('info', 'Rahul', 20);
```

Output:

```
PS C:\Users\Admin\Desktop\project node> node event.js
Name: Rahul
Age: 20
```

Practical No: 15

Q15) Create an event listener that runs only once using the once()method and verify that it doesn't execute on second emit.

Program Analysis:

The once() method creates a listener that executes only one time. When the "info" event is emitted for the first time, the listener runs and displays the message. On the second emit(), the listener does not execute because it has already been removed.

Program:

```
const EventEmitter = require('events');
const emitter = new EventEmitter();
emitter.once('info', () => {
   console.log('Info event executed');
});
emitter.emit('info');
emitter.emit('info');
```

Output:

```
PS C:\Users\Admin\Desktop\project node> node once .js
Info event executed
```

Practical No: 16

Q16)Write a Node.js program to connect to MongoDB and insert a single document into a “students”collection.

Program Analysis:

The program uses the MongoDB Node.js driver to connect to a local MongoDB server. It selects the college database and the students collection. A student document containing name, age, and course is created and inserted using insertOne(). After successful insertion, the program displays a confirmation message and the generated document ID.

Program:

```
const { MongoClient } = require("mongodb");
const url = "mongodb://localhost:27017";
const client = new MongoClient(url);
async function main() {
  try {
    await client.connect();

    const db = client.db("college");
    const students = db.collection("students");

    const student = {
      name: "Rahul",
      age: 20,
      course: "BCA"
    };

    const result = await students.insertOne(student);

    console.log("Student inserted successfully");
    console.log("Inserted ID:", result.insertedId);
  } catch (error) {
    console.log("Error:", error);
  } finally {
    await client.close();

}
```

```
}

main();
```

Output:

1) C:\Users\Admin\Desktop\project node>npm install mongodb

```
up to date, audited 19 packages in 1s
2 packages are looking for funding
 run `npm fund` for details
 found 0 vulnerabilities
```

2) PS C:\Users\Admin\Desktop\project node>node student.js

Student inserted successfully